



UNSW
SYDNEY

COMP4141

Theory of Computation

Lecture 17: Probabilistic Complexity Classes

Administrivia

- Assignment 4 available today, due **Thursday 28th April, 12:00 (AEST)**
- No lecture on Monday; Tutorials as scheduled.
- Consultation tonight cancelled
- Typo in Tutorial 8, Q2

Where next?

So far:

- Classical complexity classes: **P**, **NP**, **coNP**, **PSPACE**, **L**, ...
- Membership of the classes provides an upper bound on the resources required
- Hardness (or completeness) provides a lower bound
- Tractable (**P**, **NL**, **L**) vs Intractable (**NP**-complete, **PSPACE**-complete)

Question

How can we deal with intractable problems?

Dealing with the intractable

Classical: Consider simplifications (e.g. planar graphs, DAGs, 2-CNF)

Functional: Move away from decision problems (e.g. search problems, counting problems, optimization problems)

Approximation: Finding solutions that are “good enough”

Randomization: Using randomization to “probably” find a solution

Quantum: Solve using quantum computation

Parameterization: “Multi-dimensional” complexity analysis

Some of these topics are covered in COMP6741.

Non-classical complexity expands on what is tractable and what is intractable. It often requires introducing different types of reductions,

Outline

Randomized algorithms

Probabilistic Turing Machines

Bounded Probability

RP & ZPP

Relation to the Polynomial Hierarchy

Randomized algorithms: Motivation

- Randomization can let us “stumble upon” a solution quickly
- Average-case complexity is a practical measure
- Techniques such as derandomization, solution existence can assist classical algorithms
- Does randomization give any more computation power?

Randomized algorithms

Many randomized algorithms fall into one of these classes:

	ACCEPT	REJECT	Running time
Monte Carlo (2-sided)	$\Pr > \frac{1}{2}$	$\Pr > \frac{1}{2}$	Finite
Monte Carlo (1-sided)	$\Pr = 1$	$\Pr > \frac{1}{2}$	Finite
Las Vegas	$\Pr = 1$	$\Pr = 1$	Expected finite

Outline

Randomized algorithms

Probabilistic Turing Machines

Bounded Probability

RP & ZPP

Relation to the Polynomial Hierarchy

Probabilistic Algorithms

Definition

A *probabilistic TM (PTM)* M is a type of NTM in which each non-deterministic step is called a *coin-flip step* and has two legal next moves.

The *probability* of branch b of M 's computation on input w is

$$\Pr[b] = \frac{1}{2^k}$$

where k is the number of coin-flip steps that occur on b .

The probability that M accepts/rejects w is

$$\Pr[M \text{ accepts } w] = \sum_{b \text{ accepts } w} \Pr[b]$$

$$\Pr[M \text{ rejects } w] = 1 - \Pr[M \text{ accepts } w]$$

Probabilistic Polynomial Time (PP)

Definition

Say that PTM M *probabilistically recognises* A if it runs in polynomial time and

- ❶ $w \in A$ implies $\Pr[M \text{ accepts } w] > 1/2$, and
- ❷ $w \notin A$ implies $\Pr[M \text{ rejects } w] > 1/2$.

PP is the class of languages that are probabilistically recognised by polynomial time PTMs

PP Example

Example

Consider the following algorithm for SAT:

Given φ with n variables:

- (with probability $1/2 - 1/2^{n+1}$) *ACCEPT*,
- (with probability $1/2 + 1/2^{n+1}$) randomly generate an assignment π ; if $\pi \models \varphi$, *ACCEPT*, else *REJECT*

If $\varphi \notin \text{SAT}$, then $P(\text{ACCEPT}) = 1/2 - 1/2^{n+1} < 1/2$. If $\varphi \in \text{SAT}$,

$$\begin{aligned} P(\text{ACCEPT}) &\geq (1/2 - 1/2^{n+1}) + (1/2 + 1/2^{n+1}) \cdot 1/2^n \\ &= 1/2 + 1/2^{2n+1} \\ &> 1/2 \end{aligned}$$

Theorem

$\text{NP} \subseteq \text{PP} \subseteq \text{PSPACE}$

Proof.

$\text{NP} \subseteq \text{PP}$, follows from the previous example

For $\text{PP} \subseteq \text{PSPACE}$, simulate and count the number of accepting runs. □

Outline

Randomized algorithms

Probabilistic Turing Machines

Bounded Probability

RP & ZPP

Relation to the Polynomial Hierarchy

Bounded-error Probabilistic Polynomial Time (BPP)

Definition

For $\epsilon \in [0, \frac{1}{2})$ PTM M recognises A with error probability ϵ if

- ① $w \in A$ implies $\Pr[M \text{ accepts } w] \geq 1 - \epsilon$, and
- ② $w \notin A$ implies $\Pr[M \text{ rejects } w] \geq 1 - \epsilon$.

BPP is the class of languages that are recognised by polynomial time PTMs with an error probability of $\frac{1}{3}$.

Amplification Lemma

NB

*The definition of **BPP** is robust w.r.t. the choice of error probability; $\frac{1}{3}$ is just one convenient possibility in $[0, \frac{1}{2})$.*

Lemma

*Let $\epsilon \in [0, \frac{1}{2})$ and let $p(n) > 0$ be a polynomial.
If polynomial time PTM M recognises A with error probability ϵ
then there is another polynomial time PTM M' that also
recognises A with error probability $\exp(-p(n))$.*

Chernoff Bounds

A little probability theory:

Write $E(X)$ for the expectation (average) of a random variable X .

Theorem

Let $X_1, \dots, X_n$ be independent random (boolean) variables so that $\Pr(X_i = 1) = p_i$ and $\Pr(X_i = 0) = 1 - p_i$.

Let $X = X_1 + \dots + X_n$ and let $\mu = E(X)$. Then for $\delta \in (0, 1]$,

$$\Pr(X \leq (1 - \delta)\mu) < \exp\left(-\frac{\delta^2}{2}\mu\right)$$

and for $\delta > 0$,

$$\Pr(X \geq (1 + \delta)\mu) < \exp\left(-\frac{\delta^2}{2 + \delta}\mu\right)$$

Example

Example

We toss a fair coin n times. Let $X_i = 1$ when the i -th toss lands heads, else $X_i = 0$. The total number of heads is

$$X = X_1 + \dots + X_n.$$

The expected number of heads is

$$\mu = E(X) = E(X_1) + \dots + E(X_n) = n/2$$

What is the probability that we get $n/4$ or fewer heads? Here $\delta = 1/2$, so

$$\begin{aligned}\Pr(X \leq (1 - \delta)\mu) &< \exp(-\mu\delta^2/2) \\ &= \exp(-(n/2) \cdot (1/2)^2 \cdot (1/2)) \\ &= \exp(-n/16)\end{aligned}$$

Amplification Lemma

Proof.

Let M^r be the machine that runs M for r times on the input word and returns the outcome of a majority vote among the r outcomes.

Intuition: for a large enough r , with high probability, the number of accepts is close to the mean, i.e., roughly

- $(1 - \epsilon)r > r/2$ accepts in case $w \in A$ and
- $(1 - \epsilon)r > r/2$ rejects in case $w \notin A$

So the answer of the majority vote is correct with high probability, increasing as we increase r . For exact numbers, we calculate....

Amplification Lemma

Proof.

Let $X_i = 1$ if the i -th run accepts and $X_i = 0$ otherwise.

Let $X = \sum_{i=1..r} X_i$, i.e., the number of accept votes.

Suppose $w \in A$ (we can argue similarly for $w \notin A$). Then

$$\begin{aligned}\mu &= E(X) \\ &= \sum_{i=1..r} E(X_i) \\ &\geq \sum_{i=1..r} (1 - \epsilon) \\ &= r(1 - \epsilon)\end{aligned}$$

Amplification Lemma

Proof.

Thus, subject to some constraints on δ, r (derived later)

$$\begin{aligned} P(M^r \text{ rejects } w) &= P(X \leq r/2) \\ &\leq P(X \leq (1 - \delta)\mu) && (1) \\ &\leq \exp\left(-\frac{\delta^2 \mu}{2}\right) && \text{By Chernoff} \\ &\leq \exp(-p(n)) && (2) \end{aligned}$$

Amplification Lemma

Proof.

For (1), we need $r/2 \leq (1 - \delta)\mu$, i.e., $\frac{r}{2(1-\delta)} \leq \mu$.

Since $r(1 - \epsilon) \leq \mu$, it suffices to have $\frac{r}{2(1-\delta)} \leq r(1 - \epsilon)$, i.e., $\frac{1}{2(1-\delta)} \leq (1 - \epsilon)$, which is equivalent to $\delta \leq 1 - \frac{1}{2(1-\epsilon)}$.
(Equality will do.)

For (2), we need $p(n) \leq \frac{\delta^2}{2}\mu$, i.e., $\frac{2p(n)}{\delta^2} \leq \mu$.

Since $r(1 - \epsilon) \leq \mu$, it suffices to have $\frac{2p(n)}{\delta^2} \leq r(1 - \epsilon)$,

i.e. $\frac{2p(n)}{\delta^2(1-\epsilon)} \leq r$. Again, equality will do, so we have that r is polynomial in n (since δ, ϵ are constants). It follows that M^r runs in polynomial time. □

Primes

Since 2002, we **know** that

$$\text{PRIMES} = \{ n \mid n \text{ is a prime number in binary} \}$$

is in **P** (e.g. $O(\log^{10.5} n)$), contrary to what many had believed till then.

We'll study a (relatively simpler) proof of $\text{PRIMES} \in \text{BPP}$.

This requires a tiny bit of good old-fashioned number theory.

Let $\mathbb{Z}_p = \{0, \dots, p-1\}$ and $\mathbb{Z}_p^+ = \mathbb{Z}_p \setminus \{0\}$.

Theorem (Fermat's little theorem, 1640)

If p is a prime and $a \in \mathbb{Z}_p^+$ then $a^{p-1} \equiv 1 \pmod{p}$.

Proof.

Consider the sequence $a, 2a, 3a, \dots, (p-1)a$.

None is divisible by p . Moreover, if $ia = ja \pmod p$, then p divides $(i-j)a$, so in fact p divides $i-j$, so $i = j \pmod p$.

Thus, mod p , the sequence is a rearrangement of $1, 2, \dots, p-1$ and

$$\begin{aligned} a^{p-1}(p-1)! &\equiv a \cdot 2a \cdot 3a \cdot \dots \cdot (p-1)a \pmod p \\ &\equiv 1 \cdot 2 \cdot \dots \cdot (p-1) \pmod p \\ &\equiv (p-1)! \pmod p \end{aligned}$$

Since $(p-1)!$ is coprime with p , we can cancel this term, and get

$$a^{p-1} \equiv 1 \pmod p$$



Definition

p passes the *Fermat test at a* if $a^{p-1} \equiv 1 \pmod{p}$.

A *pseudoprime* is a number p that passes Fermat tests for all smaller co-primes a .

If p isn't pseudoprime it fails at least half of its Fermat tests.

A probabilistic algorithm for pseudoprimality would, on input p ,

- 1 Select $a_1, \dots, a_k \in \mathbb{Z}_p^+$ randomly.
- 2 Compute $a_i^{p-1} \pmod{p}$ for each i .
- 3 If all computed values are 1, accept, otherwise, reject.

Parameter k affects the error probability: it is at most $\frac{1}{2^k}$.

Pseudo-primes that aren't primes e.g.

561, 1105, 1729, 2465, ...

are called **Carmichael numbers**. They are quite rare, but (deep result) there are an infinite number of them.

What can we do about them?

Say that q is a square root of 1 modulo p if $q^2 \equiv 1 \pmod{p}$.

If p is prime then only ± 1 are square roots of 1 modulo p .

For many composite numbers, including all the Carmichael numbers, 1 has 4 or more such square roots.

Examples

$\pm 1, \pm 8$ are the square roots of 1 modulo 21.

$\pm 67, \pm 188, \pm 254$ are the square roots of 1 modulo 561.

Could we test for these square roots?

After passing the Fermat test $a^{p-1} \equiv 1 \pmod{p}$ we deduce that the number

$$a^{(p-1)/2} \pmod{p}$$

is a square root of 1 modulo p .

We continue to halve the exponent as long as it is even, checking whether the first different value is -1 . If it is a different number, then 1 has a square root modulo p that is different from ± 1 , hence p is not prime.

BPP Algorithm for PRIMES

PRIME = "On input p :

- ➊ If p is even, *accept* if $p = 2$; otherwise, *reject*.
- ➋ Let $s, t, h \in \mathbb{N}$ such that $s \cdot t = p - 1$ where s is odd and $t = 2^h$.
- ➌ Select $a_1, \dots, a_k \in \mathbb{Z}_p^+$ randomly.
- ➍ For each $1 \leq i \leq k$:
 - ➊ Compute $a_i^{p-1} \bmod p$ and *reject* if different from 1.
 - ➋ Compute the $b_{i,j} = a_i^{s \cdot 2^j} \bmod p$ for $0 \leq j \leq h$.
 - ➌ *Reject* if for the greatest j with $b_{i,j} \neq 1$ also $b_{i,j} \neq -1$.
- ➎ *Accept.*"

If p is a prime number then $\Pr[\text{PRIME accepts } p] = 1$.

If p is an even composite number then $\Pr[\text{PRIME accepts } p] = 0$.

If p is an odd composite number then $\Pr[\text{PRIME accepts } p] \leq \frac{1}{2^k}$.

For details see [Sipser2006]. We also use that modular exponentiation is in **P** for

Theorem

PRIMES $\in$ **BPP**.

Outline

Randomized algorithms

Probabilistic Turing Machines

Bounded Probability

RP & ZPP

Relation to the Polynomial Hierarchy

RP

PRIME has an interesting property: its error is *one-sided*. Any rejected number must be composite (i.e. the error probability when rejecting is 0) whereas there is a small probability that an accepted number is not prime.

Definition

RP is the class of languages that are recognisable by probabilistic polynomial time TMs where inputs in the language are accepted with a probability of at least $\frac{1}{2}$ and inputs not in the language are rejected with a probability of 1.

$$\mathbf{coRP} = \{\bar{A} \mid A \in \mathbf{RP}\} \quad \mathbf{ZPP} = \mathbf{RP} \cap \mathbf{coRP}$$

Corollary

$$\mathbf{COMPOSITES} = \overline{\mathbf{PRIMES}} \in \mathbf{RP}.$$

Monte Carlo

Definition

Call a probabilistic TM *Monte Carlo* if it accepts any given word w with probability of either 0 (i.e. rejection is guaranteed) or $> \frac{1}{2}$.

Corollary

RP is the set of languages of **P**-time Monte Carlo TMs.

It is an open problem whether $\mathbf{RP} \stackrel{?}{=} \mathbf{coRP}$.

Corollary

- ① $\mathbf{RP} \subseteq \mathbf{BPP}$
- ② $\mathbf{RP} \subseteq \mathbf{NP}$ (and hence $\mathbf{coRP} \subseteq \mathbf{coNP}$)

Las Vegas algorithms

Definition

Call a probabilistic TM *Las Vegas* if it accepts any given word w with probability of either 0 or 1 with *expected* running time in **P**.

Theorem

ZPP is the set of languages of Las Vegas TMs.

Proof.

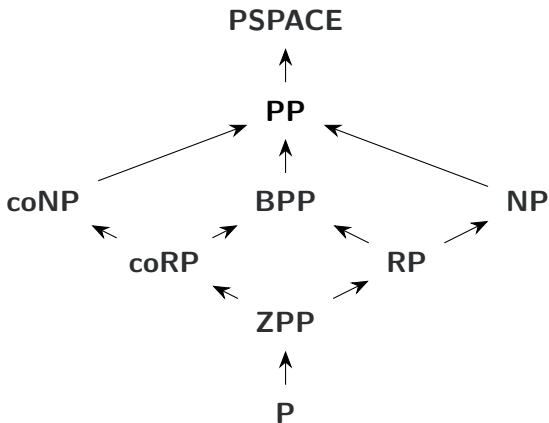
$\subseteq$: Repeat the following as long as necessary:

- Run **RP** algorithm – ACCEPT if it ACCEPTS, otherwise:
- Run **coRP** algorithm – REJECT if it REJECTS, otherwise repeat.

RP $\supseteq$: Run Las Vegas TM for twice the expected time, REJECT if no result has been computed ($\Pr < \frac{1}{2}$ by Markov's inequality)



Relationships



Many believe that $\mathbf{BPP} = \mathbf{P}$, but it is not even known if $\mathbf{BPP} \subseteq \mathbf{NP}$.

Outline

Randomized algorithms

Probabilistic Turing Machines

Bounded Probability

RP & ZPP

Relation to the Polynomial Hierarchy

Relation to the Polynomial Hierarchy

Theorem

$$\mathbf{BPP} \subseteq \Sigma_2\mathbf{P} \cap \Pi_2\mathbf{P}$$

Proof (sketch).

- **BPP** is closed under complement, so it suffices to show $\mathbf{BPP} \subseteq \Sigma_2\mathbf{P}$.
- For $L \in \mathbf{BPP}$, take a PTM that accepts L with high probability
- On input x , runs correspond to sequences of coin-flips



Relation to the Polynomial Hierarchy

Theorem

$$\text{BPP} \subseteq \Sigma_2\text{P} \cap \Pi_2\text{P}$$

Proof (sketch).

Algorithm idea:

- Non-deterministically guess a (polynomial number of) “variants” of L :
 - If $x \in L$ then every sequence of coin flips will be accepted by at least one variant
 - If $x \notin L$ then there exists a sequence of coin flips that is not accepted by any variant
- Universally guess a sequence of coin flips to decide if $x \in L$



$$\mathbf{BPP} \subseteq \Sigma_2\mathbf{P}$$

Given $L \in \mathbf{BPP}$, let M be a PTM that:

- Runs in time $p(n)$ on input size n
- Performs $k \leq p(n)$ probabilistic steps in any branch
- Accepts $x \in L$ with probability $> 1 - \frac{1}{2^n}$
- Rejects $x \notin L$ with probability $> 1 - \frac{1}{2^n}$

Observe: M is a NDTM that runs in time $p(n)$ that, on input $x \in L$ has at least $2^{p(n)}(1 - \frac{1}{2^n})$ accepting runs, and on input $x \notin L$ has at most $\frac{2^{p(n)}}{2^n}$ accepting runs.

$$\text{BPP} \subseteq \Sigma_2\text{P}$$

For $X \subseteq [1, k]$ define $M[X]$ to be the PTM that behaves as M except that it swaps the outcomes of the i -th coin-flips, for $i \in X$.

Example

If $HHH \dots$ yields an accepting run of M on x , then $THHH \dots$ yields an accepting run of $M[\{1\}]$

Claim

There exists k sets $X_1, \dots, X_k$ such that for every input x :

- *If $x \in L$ then for every sequence of k coin flips, there exists $j \leq k$ such that the sequence is an accepting run on $M[X_j]$*
- *If $x \notin L$ then there exists a sequence of k coin flips which is not an accepting run on $M[X_j]$ for any j*

$$\mathbf{BPP} \subseteq \Sigma_2\mathbf{P}$$

$\Sigma_2\mathbf{P}$ algorithm:

On input x :

- Non-deterministically guess the X_i ($k \leq p(n)$ sets of size at most k)
- Universally guess a sequence of k coin flips ($k \leq p(n)$ choices)
- Check if any of the $M[X_i]$ accepts x with the given sequence of coin flips (At most $k \leq p(n)$ simulations of $p(n)$ steps)